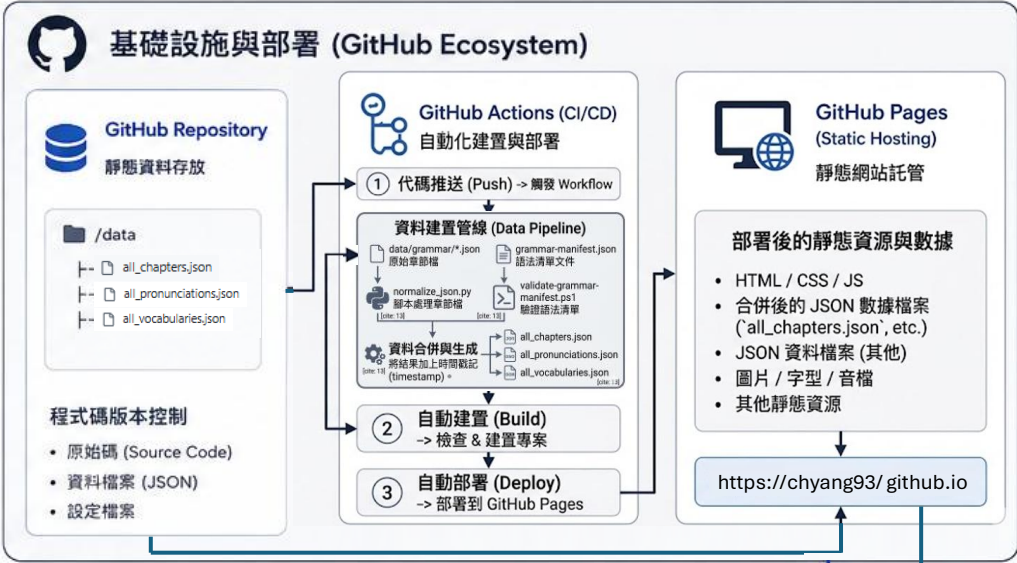


系統架構圖 (System Architecture)

韓文學習 PWA 應用程式架構



取得靜態資源 (Fetch Assets)

雙向資料同步 (Data Sync)

- 使用者認證
- 進度上傳
- 資料下載
- 即時同步



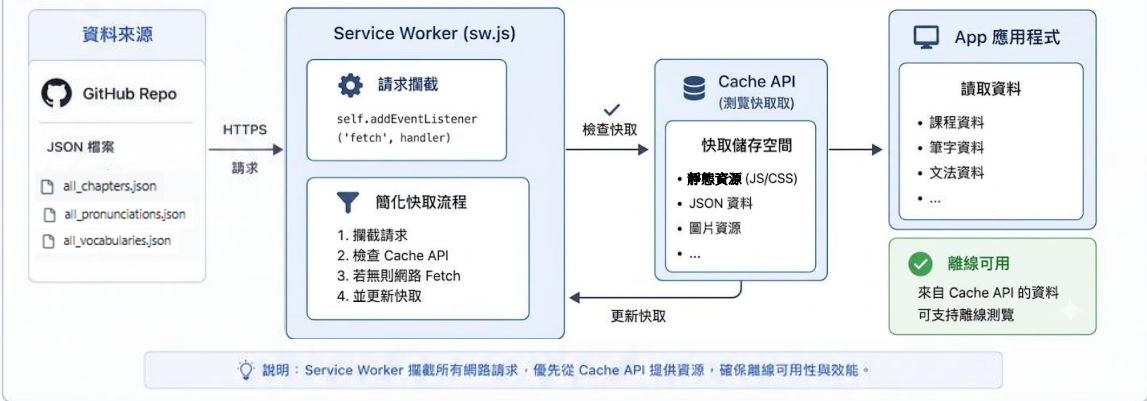
核心特性



資料流程圖 (Data Flow & Offline Sync)

學習資料流向與離線快取 / 同步機制總覽

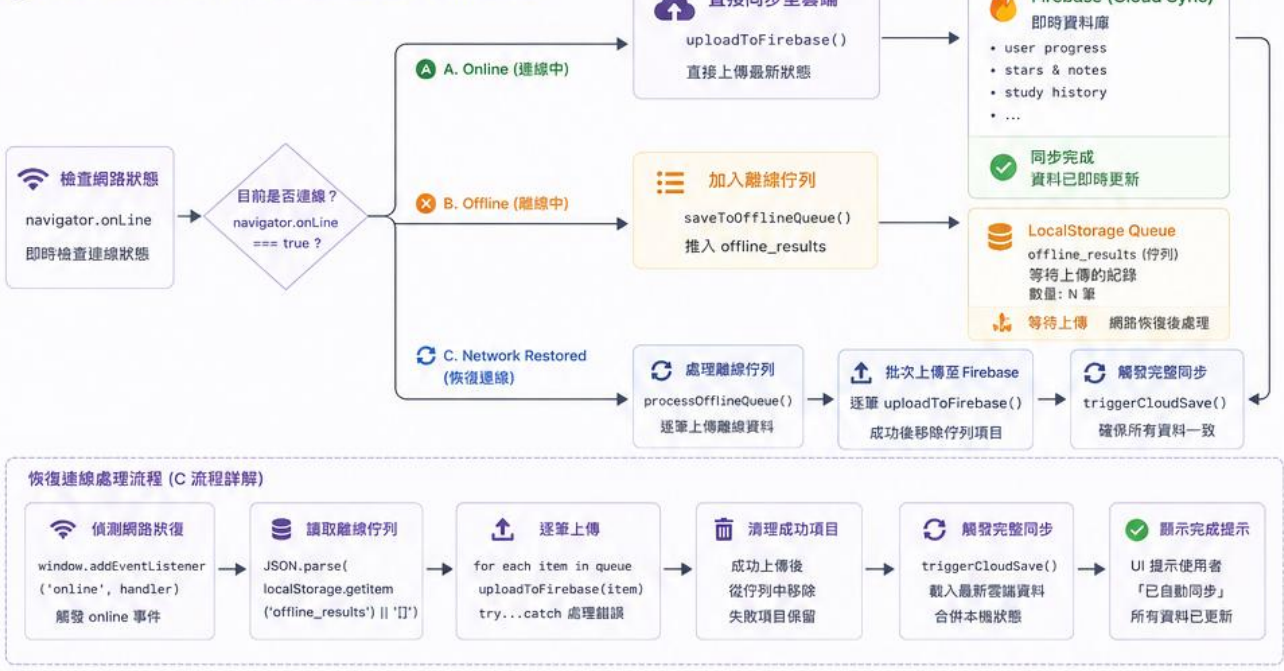
1 階段一：資料獲取 (Fetching Data)



2 階段二：進度產生與儲存 (State Persistence)



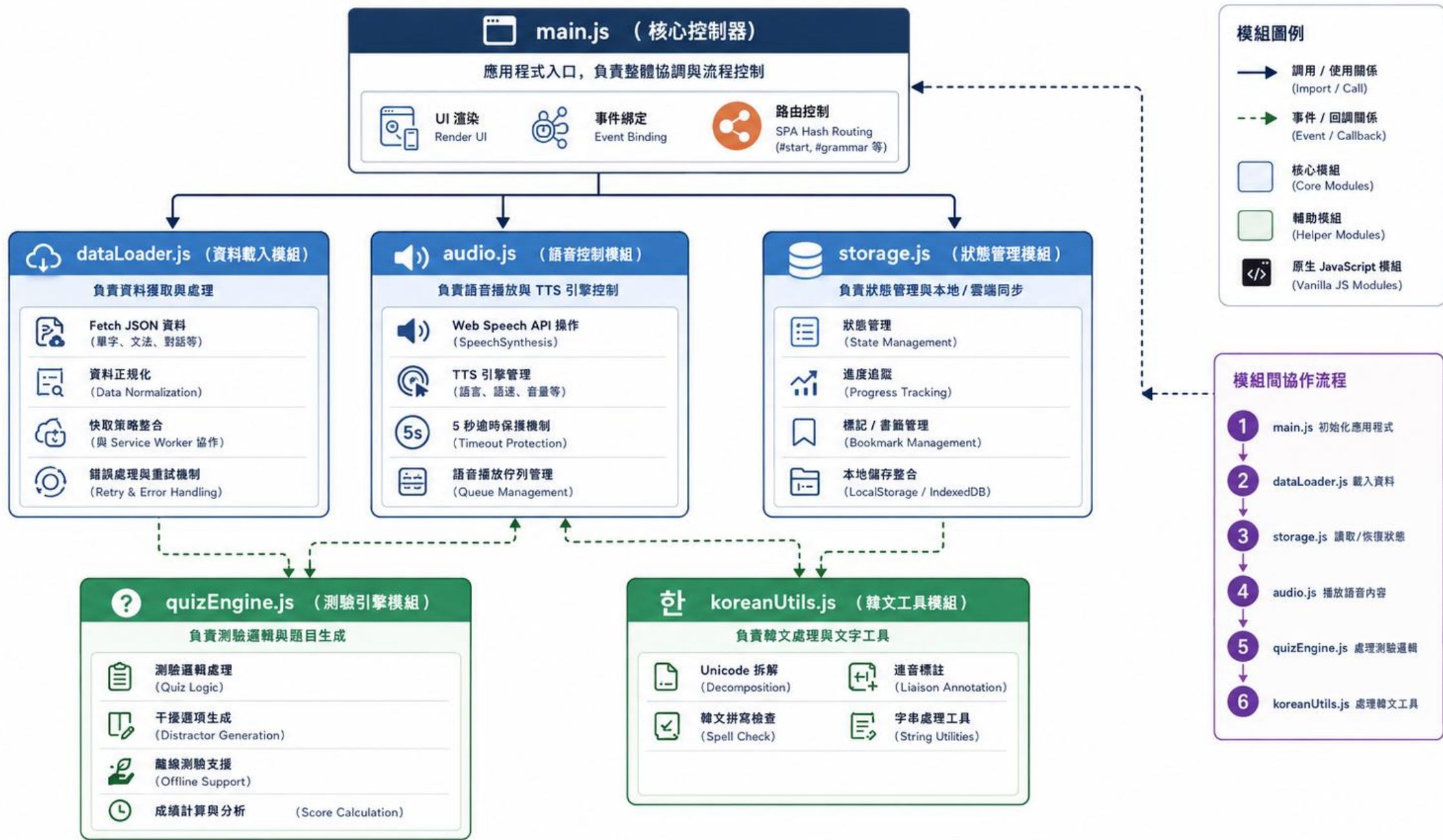
3 階段三：網路狀態判斷 (Network Decision Branch)



資料流向總結

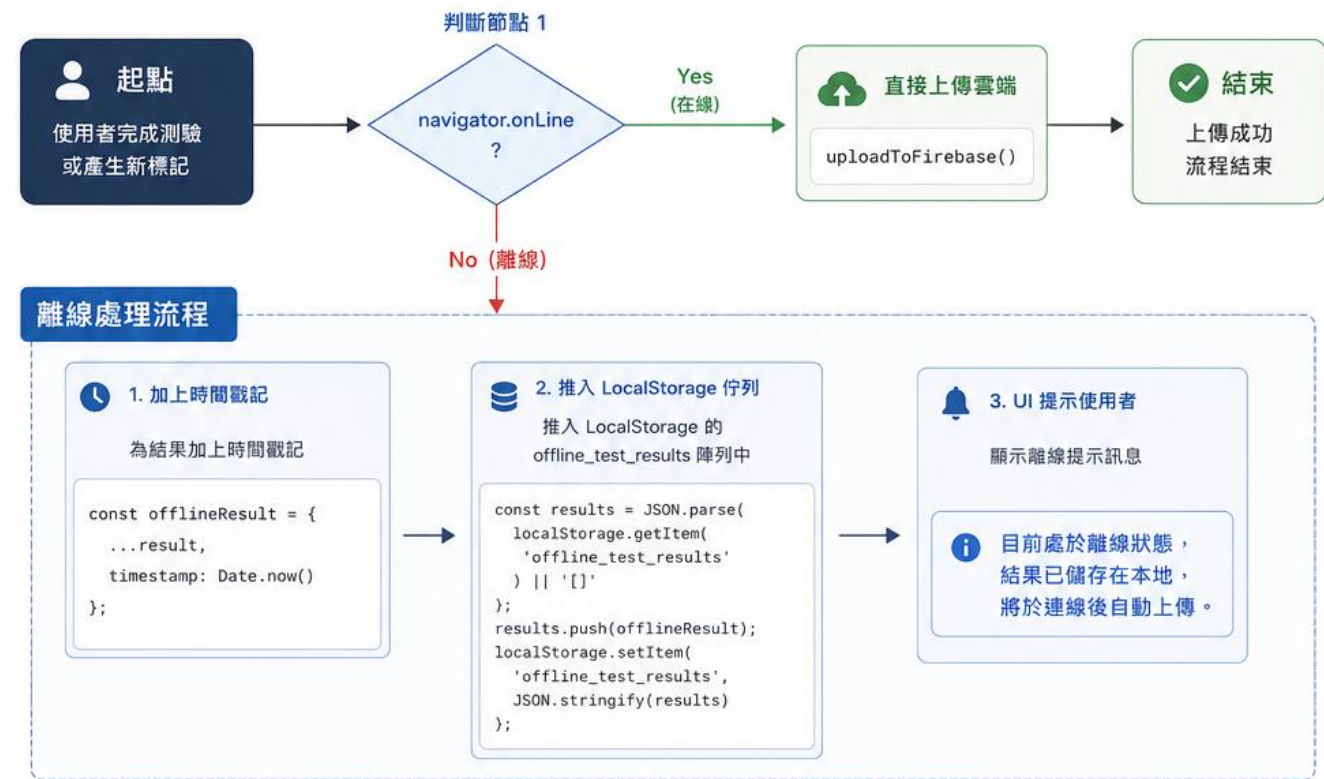


JS 模組架構圖 (JS Modules & Core Functions)

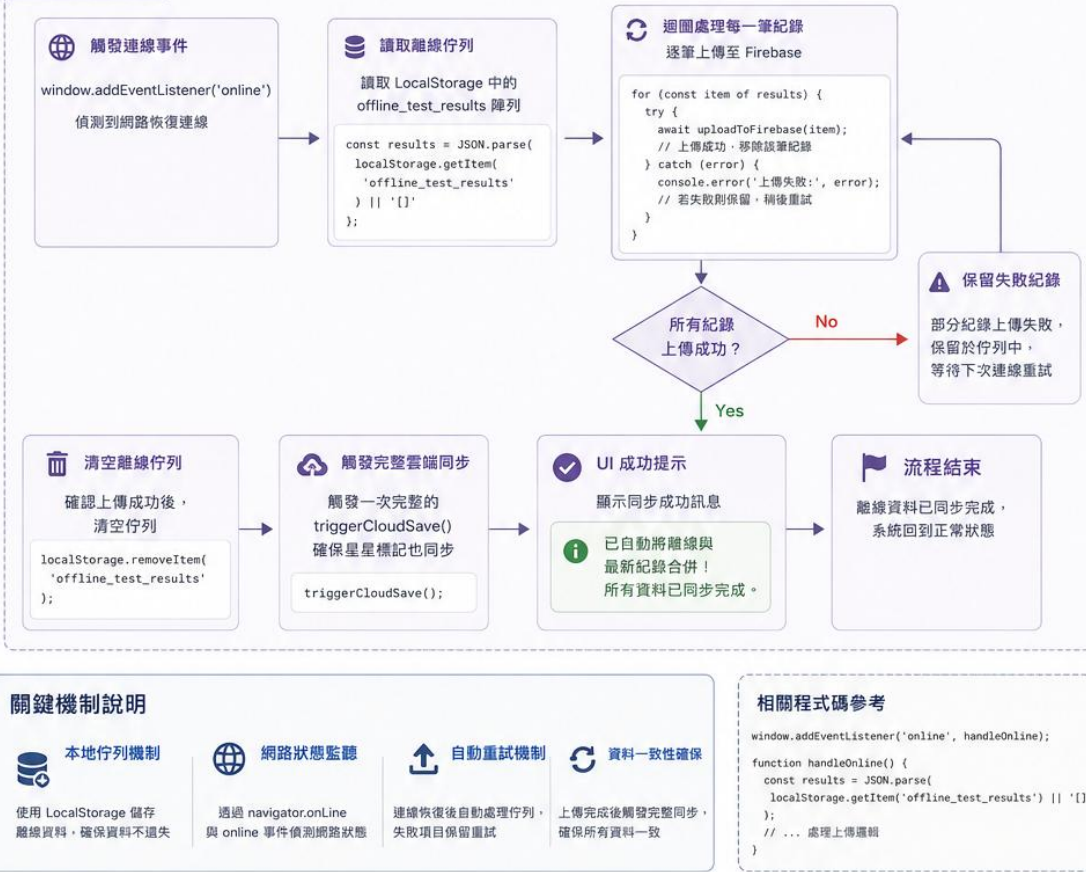


離線同步流程圖 (Offline Sync Flowchart)

斷網存檔與連線自動上傳的完整邏輯

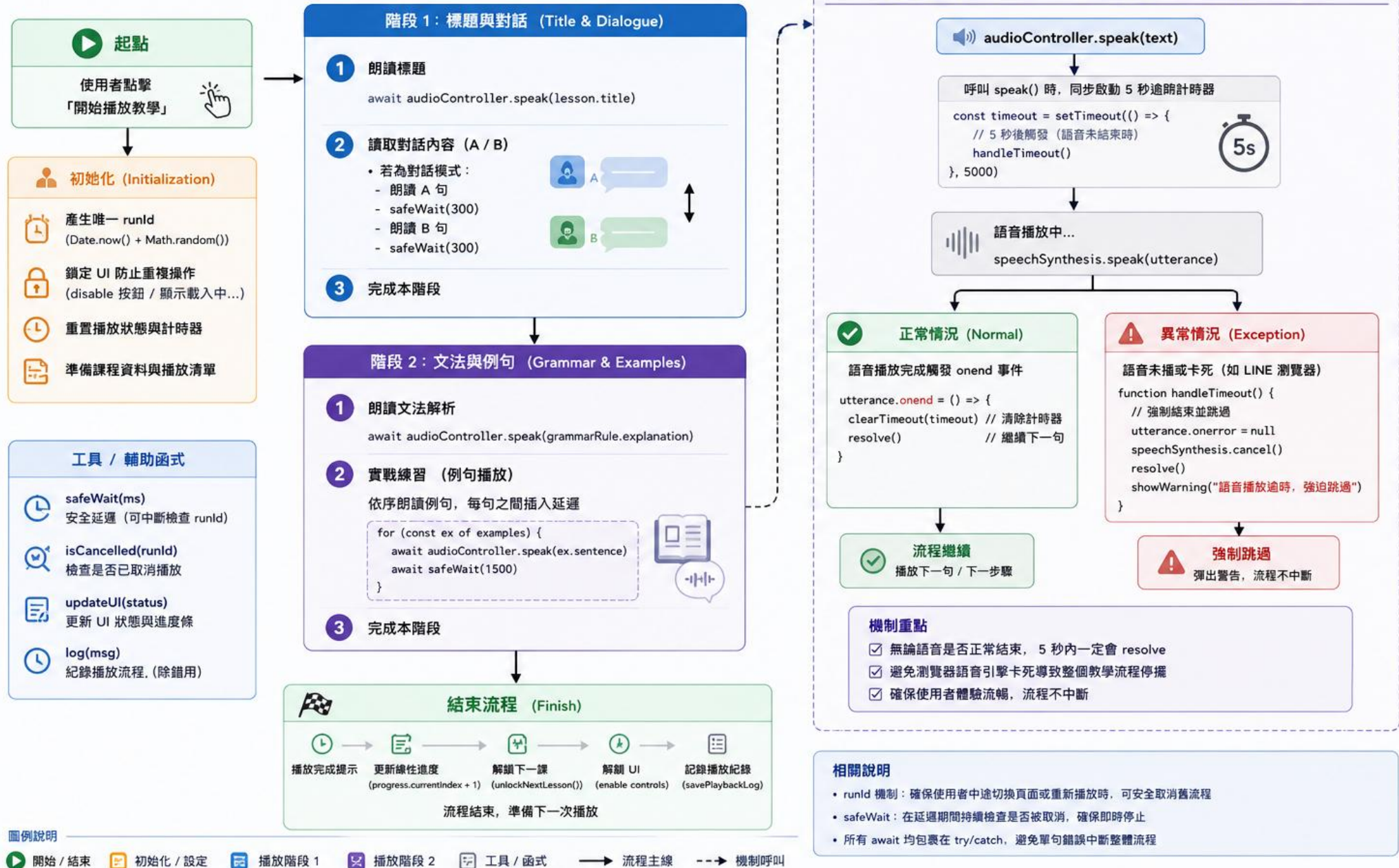


恢復連線流程



教學播放邏輯圖 (Audio Tutorial Logic)

playFullLesson() 自動化教學流程與防卡死機制



資料結構圖 (Data Structure)

靜態資料 (JSON) × 動態狀態 (State) × 資料夾功能 (Folder) 之關聯架構



靜態資料層 (JSON)

從 GitHub Pages 載入的靜態資料

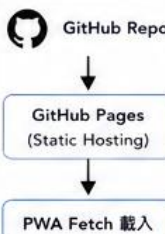
all_vocabularies.json (單字資料)

```
[
  {
    "id": "V001",
    "ko": "학교",
    "zh": "學校",
    "part": 1,
    "level": "TOPIK I",
    "tags": ["場所", "教育"]
  },
  {
    "id": "V002",
    "ko": "학생",
    "zh": "學生",
    "part": 1,
    "level": "TOPIK I",
    "tags": ["人物", "教育"]
  },
  ...
]
```

欄位說明

id	唯一識別碼
ko	韓文
zh	中文翻譯
part	所屬章節
level	難度等級
tags	分類標籤

資料來源



動態狀態層 (Global State)

使用者學習狀態 (儲存在 LocalStorage 並同步 Firebase)

progress (Global State)

```
{
  "currentLessonId": "L15",
  "learnedVocab": ["V001", "V002"],
  "bookmarkedVocab": ["V001"],
  "testResults": {},
  "studyStats": {},
  "updatedAt": 1712345678901
}
```

主要狀態說明

- learnedVocab
已學單字 ID 陣列
- bookmarkedVocab
已標記單字 ID 陣列
- testResults
測驗成績記錄
- studyStats
學習統計資訊

儲存與同步



資料夾擴充層 (v5.4 - Folder)

使用者自定義資料夾功能

folders (資料夾清單)

```
[
  {
    "id": "f_1712345678901_ab12",
    "name": "必考單字",
    "createdAt": 1712345678901,
    "color": "#FFB74D",
    "sortOrder": 1
  },
  {
    "id": "f_1712345678901_cd34",
    "name": "TOPIK 中級",
    "createdAt": 1712345678902,
    "color": "#64B5F6",
    "sortOrder": 2
  }
]
```

欄位說明

id	資料夾唯一 ID
name	資料夾名稱
createdAt	建立時間戳
color	資料夾顏色
sortOrder	排序順序

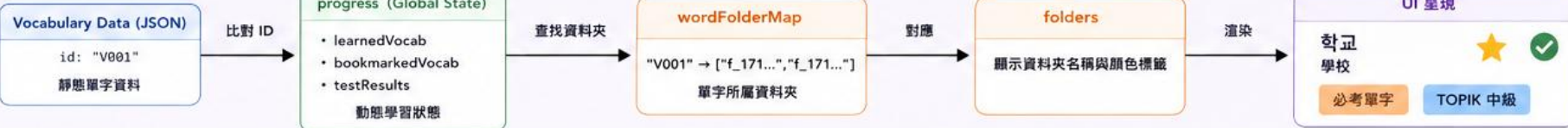
wordFolderMap (單字 → 資料夾對照表)

```
{
  "V001": ["f_1712345678901_ab12", "f_1712345678901_cd34"],
  "V002": ["f_1712345678901_ab12"],
  "V003": ["f_1712345678901_ef56", "f_1712345678901_ab12"],
  "V004": []
}
```

用途說明

將單字 ID 映射到多個資料夾 ID，支援一對多關聯

關聯關係 (Relationship)



渲染單字庫時的資料流程 (Rendering Flow)



圖例 (Legend)

- 靜態資料 (JSON)
- 動態狀態 (State)
- 資料夾功能 (Folder)
- UI / 流程 (Process)

設計重點

- 單字 ID 為核心關聯鍵 (Single Source of Truth)
- 學習狀態與資料夾完全解耦，彈性擴充
- 支援單字屬於多個資料夾 (一對多)
- 離線優先：LocalStorage → 線上同步 Firebase
- 高效查詢：透過 Map 結構快速定位資料夾關聯

雲端衝突對比畫面 (Conflict UI)

跨裝置同步衝突偵測與使用者選擇機制

 本機裝置 (Local)

本機狀態

📖 課程進度	ID: 10
Aa 已學單字	50 個
📊 成績紀錄	15 筆

 雲端 (Cloud)

雲端狀態

📖 課程進度	ID: 15
Aa 已學單字	120 個
📊 成績紀錄	45 筆



偵測到
資料不一致



發現不同裝置的紀錄不一致！

本機與雲端的資料存在差異，請選擇要採取的動作：

📖 課程進度	本機 ID	10	vs	雲端	15
Aa 已學單字	本機	50	vs	雲端	120
📊 成績紀錄	本機	15 筆	vs	雲端	45 筆



建議：若要保留最新的學習進度與資料，請選擇「下載雲端進度」



取消

保留本機進度
(將本機紀錄合併至雲端)



確定

下載雲端進度
(覆蓋此裝置)



同步演算法概念



1. 觸發同步

使用者登入或手動同步



2. 下載並比較

取得雲端資料並偵測衝突



3. 衝突處理

顯示對比畫面讓使用者選擇



4. 完成同步

更新本機與雲端狀態



技術實作機制

1. 衝突偵測：hasMeaningfulConflict()

- 比較關鍵 ID (如courseId)
- 比較陣列長度 (如單字數、成績筆數)
- 任一差異即判定為有意義的衝突

```
function hasMeaningfulConflict(local, remote) {  
  return local.courseId == remote.courseId  
    && local.vocabs.length == remote.vocabs.length  
    && local.scores.length == remote.scores.length  
}
```

2. 衝突合併：mergeStateForConflict()

- 深度合併物件與陣列
- 比業務規則合併 (去重、更新較新資料)
- 產生合併後的最終狀態

```
function mergeStateForConflict(local, remote) {  
  return {  
    ...remote,  
    vocabs: mergeVocabs(local.vocabs, remote.vocabs),  
    scores: mergeScores(local.scores, remote.scores)  
  };  
}
```

3. 使用者選擇後的流程



資料安全保護

- 同步前自動備份本機資料
- 所有合併操作皆在本機進行預覽
- 使用者明確選擇後才會執行寫入

韓文連音演算法拆解圖

annotateKoreanText() 演算法視覺化



輸入字串



Unicode 拆解



規則判斷



DOM 注入輸出

範例 1 輸入字串：안녕 (An-nyeong)

1 第一步：Unicode 拆解 (decompose 函式)

前字：안

안 U+C548
= ㅇ + ㅏ + ㄴ

初聲 (cho)	中聲 (jung)	終聲 (jong)
ㅇ	ㅏ	ㄴ
11	0	4

jong > 0 (有終聲)

後字：녕

녕 U+B155
= ㄴ + ㅕ + ㅇ

初聲 (cho)	中聲 (jung)	終聲 (jong)
ㄴ	ㅕ	ㅇ
2	6	21

cho != 11 (初聲非 ㅇ)

2 第二步：規則判斷

連音規則條件

- 前字有終聲 (jong > 0)
- 終聲不是 ㅇ (jong !== 21)
- 後字初聲為 ㅇ (cho === 11)

條件檢查結果

- ✓ jong > 0 (ㄴ)
- ✓ jong !== 21 (ㄴ != ㅇ)
- ✗ cho === 11 (ㄴ != ㅇ)

判斷結果

✗ 不符合連音條件
因後字初聲為 ㄴ，
不產生連音提示。

3 第三步：DOM 注入 (無連音處理)

輸出結果

안녕
(無特殊標記)

範例 2 輸入字串：먹어 (Meog-eo)

1 第一步：Unicode 拆解 (decompose 函式)

前字：먹

먹 U+BA39
= ㅁ + ㅑ + ㄱ

初聲 (cho)	中聲 (jung)	終聲 (jong)
ㅁ	ㅑ	ㄱ
6	4	1

jong > 0 且 jong !== 21 (有終聲且非 ㅇ)

後字：어

어 U+C5B4
= ㅇ + ㅓ + (無終聲)

初聲 (cho)	中聲 (jung)	終聲 (jong)
ㅇ	ㅓ	-
11	4	0

cho === 11 (初聲為 ㅇ)

2 第二步：規則判斷

連音規則條件

- 前字有終聲 (jong > 0)
- 終聲不是 ㅇ (jong !== 21)
- 後字初聲為 ㅇ (cho === 11)

條件檢查結果

- ✓ jong > 0 (ㅓ)
- ✓ jong !== 21 (ㅓ != ㅇ)
- ✓ cho === 11 (ㅇ == ㅇ)

判斷結果

✓ 符合連音條件
前字終聲 ㄱ 將與後字
初聲 ㅇ 發生連音。

3 第三步：DOM 注入 (產生連音提示)

輸出結果 (HTML)

ㅁ어

連音提示樣式 (CSS 範例)

```
.liaison-hint {
  color: #2A7AE2;
  border-bottom: 2px dotted #2A7AE2;
  position: relative;
}
```

底線虛線表示
可連音位置提示

演算法流程總覽 (annotateKoreanText)

```
function annotateKoreanText(text) {
  for (let i = 0; i < text.length - 1; i++) {
    const [c1, j1, jg1] = decompose(text[i]); // 前字拆解
    const [c2, j2, jg2] = decompose(text[i + 1]); // 後字拆解
    if (jg1 > 0 && jg1 !== 21 && c2 === 11) { // 連音條件判斷
      標記連音提示 (DOM 注入)
    }
  }
}
```

韓文 Jamo 索引參考

初聲 (cho)	中聲 (jung)	終聲 (jong)
ㅇ = 11	ㅏ = 0	(無終聲) = 0
ㄴ = 2	ㅑ = 4	ㅓ = 1
ㄷ = 3	ㅕ = 6	ㄴ = 4
ㅁ = 6	...	ㅇ = 21
...		...

關鍵重點

- ✓ 連音僅發生在「前字有終聲」且「後字初聲為 ㅇ」的情況
- ✓ 終聲為 ㅇ (jong = 21) 時視為無終聲，不觸發連音
- ✓ 連音提示透過 標記，提供視覺化學習輔助
- ✓ 此演算法適用於所有韓文句子，即時處理每一對相鄰音節